

Accessing and Interpreting OPC UA Event Traces based on Semantic Process Descriptions

Tom Westermann

Institute of Automation Technology,
Helmut-Schmidt-University
Hamburg, Germany
tom.westermann@hsu-hh.de

Nemanja Hranisavljevic

Fraunhofer Center for Machine Learning
Fraunhofer IOSB-INA
Lemgo, Germany
nemanja.hranisavljevic@iosb-ina.fraunhofer.de

Alexander Fay

Institute of Automation Technology,
Helmut-Schmidt-University
Hamburg, Germany
alexander.fay@hsu-hh.de

Abstract—The analysis of event data from production systems is the basis for many applications associated with Industry 4.0. However, heterogeneous and disjoint data is common in this domain. As a consequence, contextual information of an event might be incomplete or improperly interpreted which results in suboptimal analysis results. This paper proposes an approach to access a production systems' event data based on the event data's context (such as the product type, process type or process parameters). The approach extracts filtered event logs from a database system by combining: 1) a semantic model of a production system's hierarchical structure, 2) a formalized process description and 3) an OPC UA information model. As a proof of concept we demonstrate our approach using a sample server based on OPC UA for Machinery Companion Specifications.

Index Terms—Ontology, OPC Unified Architecture, Semantic Web, Cyber-Physical Production System, Formalized Process Description

I. INTRODUCTION

One vision of Industry 4.0 are cyber-physical production systems (CPPS) that are self-diagnosing, self-adapting and self-optimizing [1]. During production, these systems generate large amounts of multivariate data, which contain information about the systems' behaviour. In many applications, such as bottleneck analysis, predictive maintenance or anomaly detection, the stored data can be used to learn a model which captures the systems' behaviour. Often, these models take the form of Petri nets, Markov chains or various types of automata [2] [3].

In practice, time-annotated production data is difficult to process [4] due to numerous reasons. The system architecture of production systems is heterogeneous and consists of many different interacting systems whose components are produced by different manufacturers and may use heterogeneous data models [5]. These components often have their own data storage, which could be relational databases, log files or other formats. Across these different data silos, similar concepts might be represented in different ways. This heterogeneous and complex character of the data is one of the biggest obstacles for the exchange of knowledge in automation systems [6] and big data applications in general [7].

Another problem encountered during the analysis of event traces is the unavailability of prior knowledge. In analysis, this

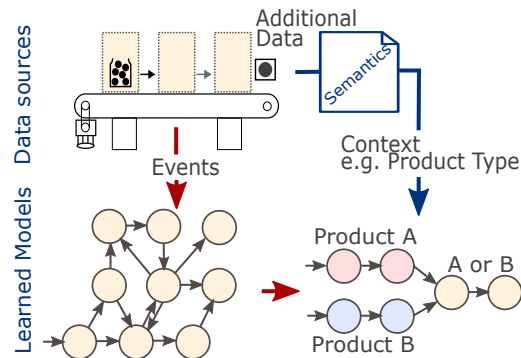


Fig. 1: Semantically enriching event logs with context and additional data may result in more accurate and understandable analysis results.

lack of information can cause "rediscovering" information that is already known but unavailable, or lead to wrong conclusions about the system's behaviour. This is depicted in Figure 1. It shows two models, one learned from the raw event stream, and the other learned from raw events and additional knowledge like the type of process and the type of product that was produced. The first model might be complex and opaque to the operator. However, it is expected that enriching the logs with information from additional data sources can improve the learned model both in terms of accuracy as well as interpretability.

Semantic Web Technologies (SWT) like ontologies, the query language *SPARQL* or knowledge graphs can provide a solution to the described problems [8]. An ontology can provide an explicit specification of the concepts and their relationships in a domain. If this conceptualization is combined with instances and their relations, this can be considered a knowledge graph. This knowledge graph can provide a way to align incompatible data that semantically refers to the same entity. With ongoing research in the field of ontology-based data access (OBDA), Semantic Web Technologies can also be used to integrate information from distinct data silos. With regard to the analysis of time-annotated event traces from CPPS, knowledge graphs also offer a way to provide formalized prior knowledge, i.e. a machine interpretable model of implicit knowledge that is otherwise not available in the analysis.

This paper proposes a knowledge graph that allows extracting time spans that correspond to the execution of production processes at a CPPS, based on a formalized process description. The extracted process time spans can then be used along with an OPC UA information model to access a time-series database. This makes it possible to retrieve event traces that correspond to these processes.

The remainder of the paper is structured as follows: Section II presents and discusses relevant research. Section III defines an ontology to access variable values from an OPC UA information model based on a formalized process description. A prototypical implementation of this is shown in Section IV and evaluated in Section V.

II. RELATED WORK

This section gives an overview of numerous works that used Semantic Web Technologies to either model information about sensors of a technical resource, or access this type of information through an OPC UA server.

Jiskovsky et al. [5] propose a semantic big data historian that uses Semantic Web Technologies to integrate sensor data from a hydroelectric power plant and combines it with additional information from other systems. It proves successful at resolving heterogeneity between different systems. However, the sensor measurements are stored inside the triple store, which can have a negative effect on the graph's size and performance. The knowledge graph does not consider possibilities for reuse.

Schiekofer et al. provide a formal mapping between OPC UA and technologies of the semantic web stack [9]. This allows for the explicit description of OPC UA semantics, which are usually only implicitly defined in the information models' documentation. The authors also offer methods for consistency checking of the OPC UA information model using a reasoner and querying the information model using SPARQL queries.

Hildebrandt et al. present a domain-expert-centric approach to ontology design of CPPS, focusing on industry standards as ontology design patterns [10]. The approach is validated on an industrial use case encompassing system structure, processes, states, and data elements and has a strong focus on ontology reuse. Access to high-velocity data that should not be stored in the ontology is not in the scope of the paper.

Kalaycı et al. use OBDA to access heterogeneous manufacturing data from a knowledge graph [11]. This knowledge graph represents the concepts and properties that are relevant for surface mounting process manufacturing, together with important domain knowledge. Using the OBDA-tool Ontop, SPARQL queries were executed to answer analytical queries about the production and failures. The approach is only applicable to relational databases, which can be accessed via SQL queries.

Xiao et al. [8] list further use cases of OBDA, spanning diverse domains including manufacturing, process mining and administration, while Ekaputra et al. [12] review OBDA approaches in multi-disciplinary engineering environments.

Steindl et al. propose a different ontology-based method to access OPC UA data through a regular SPARQL endpoint and Custom Property Functions (CPF). These CPF extend the SPARQL query evaluator of the Apache Jena Framework with custom code, that gets executed whenever the CPF gets called in a query. In order to avoid overloading the triple store, the OPC UA data is stored in a separate database and only accessed on-demand [13]. It succeeds in accessing the recorded values. Queries that rely on an additional knowledge outside the extracted OPC UA information model were not in the scope of the paper. This approach relies on OPC UA Historical Data Access, which accesses historical values through the OPC server itself. For read-only tasks, Mathias et al. recommend accessing the database directly, because passing through the OPC UA server might cause computational overhead on the PLC [14].

Steindl et al. further evaluated the query execution times of three different methods to integrate time series data into knowledge graphs [15]. The authors found that both Ontop and custom property functions were superior to storing the time series data inside the knowledge graph. If the time series data is already stored in a preexisting SQL database, they recommend Ontop. Otherwise they recommend using CPFs.

While there are many approaches to model static information about CPPS, no method of accessing segments of time-annotated sensor data based on this information exists. This work therefore aims to close the gap between the approach to ontology modeling from Hildebrandt et al. [10] and the data access to logged OPC UA data from Steindl et al. [13]. The approach from Hildebrandt et al. [10] is used to formalize prior knowledge about the structure and processes of a CPPS. Based on this process description, process-based event traces can be extracted using the approach from Steindl et al. [13].

III. ONTOLOGY OF PROCESSES, PRODUCTION SYSTEMS AND THEIR VARIABLES

As described in Section I, the analysis of CPPS event traces is hindered by semantic heterogeneity across data sources, separate data silos and unavailability of prior knowledge during analysis. To overcome these issues, this section introduces an ontology that formalizes prior knowledge about processes and their types. It is then combined with additional information on events and time series that were recorded at a CPPS from an OPC UA server.

The creation of the ontology follows the domain-expert-centric-approach to industrial ontology design outlined by Hildebrandt et al. [10], in which suitable ontology design patterns (ODP) are identified based on user-defined competency questions (CQ) that the ontology is supposed to answer.

A. Requirements and Design Patterns

The goal of the ontology is to provide semantic access to time-annotated data of CPPSs. Therefore, the first requirement is that the model needs access to the full event trace of a CPPS. This requirement can in turn be divided into two distinct competency questions.

The first competency question asks for information of which variables belong to the CPPS in question. In modern CPPSs, this information is available in the OPC UA servers information model. This model describes standardized nodes and their relationships of a server's address space. They can be modeled according to OPC UA companion specifications, which further specify models e.g. for specific industries.

Since queries should refer to individual machines, OPC UA for Machinery is a suitable companion specification to use in this work [16]. It allows clear identification of individual machines and the variables they organize. Because OPC UA for Machinery functions as a container for machine representations that follow other companion specifications (e.g. machine tools or robotics), it is not restricted to a single domain. All parts of the information model that are relevant for this use case can be translated into OWL. To achieve this, the OPC UA information models can be automatically extracted and transformed into RDF-triples [10] [13].

RQ 1:	Full Event trace of single machine	
CQ 1.1:	Which variables belong to a certain machine?	Machine x organizes variables a, b, c
CQ 1.2:	Which events and values were recorded at a specific machine in a specific timeframe?	Events x, y, z happened at Variables a, b, c
ODP:	OPC UA for Machinery + Companion Specification	

Table I: Requirement and derived competency questions for RQ1.

The second competency question refers to the extraction of variable value changes that were recorded in a given time interval. This opens up the question of data storage and access. Since these values are recorded and written at short intervals, they are not well suited to be stored inside a graph database. Multiple approaches for access to this type of data exist in the literature, among them Virtual Knowledge Graphs [11] or CPFs. For this use case a CPF was chosen following the approach from [13]. Further information on the implementation can be found in Section IV.

As described in Section I, similar production processes are expected to produce similar event traces during their execution. Therefore the access to these values should be based on the production process that was performed at the time the events occurred. To classify these production processes, an ODP based on the ISA-88 model for batch control can be used [17]. It contains three types of information: The physical model describes physical assets and their hierarchy, while the procedural control model describes abstract recipes and operations that these physical assets can perform. Additionally, it includes a process model that describes realizations of these abstract recipes and operations.

These process realizations can be described in greater detail using the Formalized Process Description according to the VDI 3682 [18]. It offers a formalized and easily understandable description mechanism. It can be used to describe which technical resource performed a process and which products/energy/information elements were used as inputs or outputs. Especially relevant to the use case in this paper would be

an information element regarding the start and end timestamp of the process. Previous works that used this approach to ontology design modeled information elements according to the data element defined in DIN EN 61360 [19] [20]. The data element contains a type description of the element as well as an instance description, which can contain the value associated with the data element.

RQ 2:	Events and time series data of machines should be accessible based on their context	
CQ 2.1:	Which events happened at a machine during a specific type of production process?	Events x, y, z happened at Variables a, b, c
CQ 2.2:	Which events happened at a machine during production of a certain product type?	Events x, y, z happened at Variables a, b, c
ODP:	ISA-88: Physical assets and process types VDI3682: Description of process instances DINEN61360: Data Elements	

Table II: Requirement and derived competency questions for RQ2.

B. Lightweight Ontology

The use of three different ODPs causes semantic heterogeneity between some concepts of the ontology that needs to be resolved. Between the ODPs of the ISA 88 and the VDI 3682, this heterogeneity is twofold. Any *Physical Asset* in the ISA 88 model that can perform a part of a *Recipe Procedure* can be considered a *Technical Resource*. Therefore, they can be considered subclasses of the broader term *Technical Resource* from the VDI 3682. Additionally, any *Process* in the ISA 88 process model can be described as a *Process Operator* from the VDI 3682. These are therefore subclasses of the *Process Operator* as well. This is visualized in Figure 2.

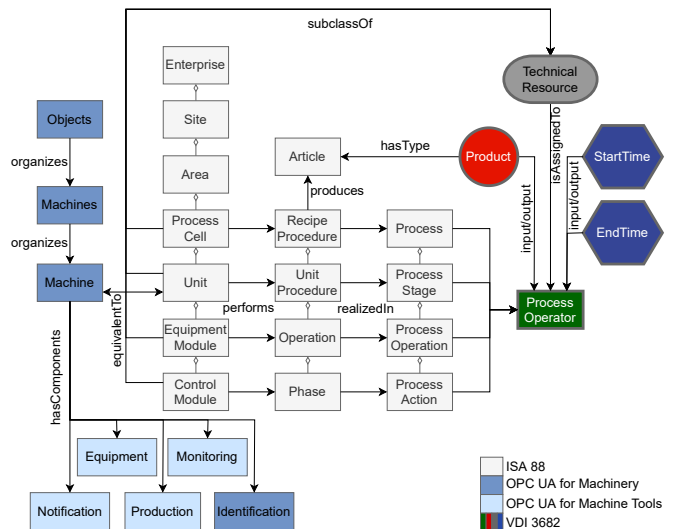


Fig. 2: Lightweight ontology of the semantic integration of ISA 88, VDI3682 and OPC UA for Machinery. Please note that both OPC UA information models are very large and were therefore truncated.

Further semantic heterogeneity exists between the OPC UA for Machinery information model and the physical model from

the ISA 88. While the ISA 88 model describes a machine with regards to the kinds of operations it can perform, the OPC UA for Machinery model describes a machine along with the data that it generates. The machine itself should however be represented in the ISA 88 model as well. Since the machines in the OPC UA for Machinery model are individually capable of performing processes, they correspond to *Units* in the ISA 88 model.

IV. USE CASE: PROCESS-BASED ACCESS TO EVENT LOGS OF AN OPC UA SERVER

In order to validate the functionality of the described approach, the ontology described in Section III was implemented and filled with data describing a machine and process hierarchy according to ISA88. The knowledge about the production facility was modeled in OWL and stored in a knowledge graph. The graph was then stored in Apache Jena¹, which is an open source framework to build applications based on Semantic Web Technologies (s. Figure 3).

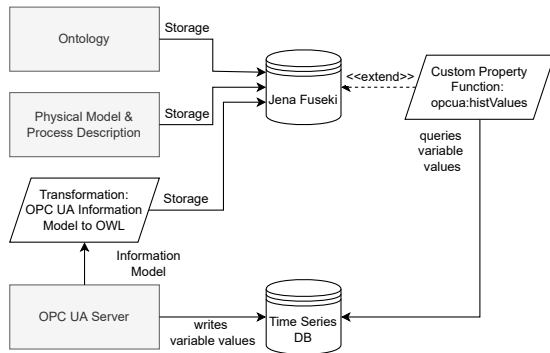


Fig. 3: Storage and access setup that was used in the validation experiment. Ontology, physical model and process description were materialized along with the OWL-representation of the OPC UA information model.

For this validation, a simulated OPC UA sample server from UMATI was used [21]. Its information model follows the OPC UA for Machinery companion specifications [16]. Inside its *Machines*-folder, it contains various machines (eg. machine tools [22], woodworking tools, robots etc.) that follow their respective OPC UA Companion Specifications. These machines were considered part of a physical model according to ISA88, i.e. a *ISA88:Unit* that can perform *ISA88:UnitProcedures*. The OPC UA information model was extracted via an automated node crawler and transformed into OWL using the Open Source Tool *Lion*².

In order to log the events and time series data from the OPC UA server, multiple approaches exist. The different events and time series values can be materialized inside the triple store. Due to the high volume and velocity of measurements however, this approach would result in inefficient query times [15] and data processing in general [5]. Because of these limitations, a dedicated time series database was favored instead.

For this task, InfluxDB³ was chosen since it is well suited for high write loads. All variable value changes from the machines on the OPC UA server were logged using the open source server agent *Telegraf*⁴ and its OPC UA plugin⁵. The time series is stored in InfluxDB according to the variable's *nodeId*, so that all information necessary for the query can be extracted from the ontology. This way, all value changes of variables that have an OPC UA type definition of *BaseDataVariableType*, *AnalogUnitRangeVariableType* or *FiniteStateVariableType* are logged in the database.

To allow direct access from the ontology, the approach by Steindl et al. [13] was modified. As in [13], a custom property function was registered as an extension in Apache Jena's SPARQL processor *ARQ*. If executed, the custom property function queries the node's variable value changes from a database for any nodes that match the graph pattern specified in the rest of the SPARQL query. The historical values can therefore be accessed by calling the custom property function *?node histValues(?time ?value ?starttime ?endtime)* with *node* being the selected node from the OPC UA information model and *histValues()* being the parametrized custom property function. For the given case, a connector to InfluxDB was implemented that automatically queries a variables historical data between two timestamps using InfluxDB's query language.

The knowledge graph was filled with descriptions of processes using an OWL representation of the formalized process description. These descriptions contain information on the technical resource that was assigned to the process, the ISA88-type of process that was executed, as well as its inputs, outputs and start- and end times. This information could usually be found in an MES system. The lower part of Figure 4 shows some of the process information that is available.

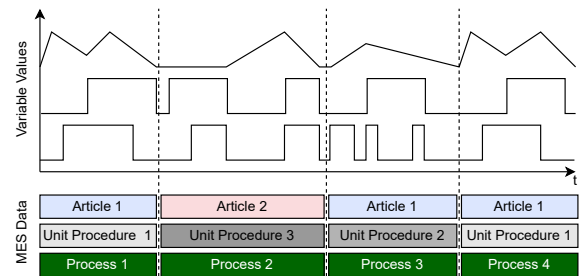


Fig. 4: Data access principle for a single machine: Processes can be selected based on various criteria. Using the start and end time of the processes, individual event traces can be extracted.

Using this information, it is possible to select process instances based on various selection criteria e.g. the article that was produced or the procedure that was realized in this process via SPARQL queries. Since the technical resource of the process is known, the variables of that resource can be extracted from the OPC UA information model. Variables and timestamps are then passed to the Custom Property Function

¹<https://jena.apache.org/>

²<https://github.com/hsu-aut/lion>

³<https://portal.influxdata.com/downloads/>

⁴<https://www.influxdata.com/time-series-platform/telegraf/>

⁵<https://www.influxdata.com/integration/opcua/>

opcua:histValues(), which queries the variable values from the time series database. Figure 5 describes the information flow for one of these queries.

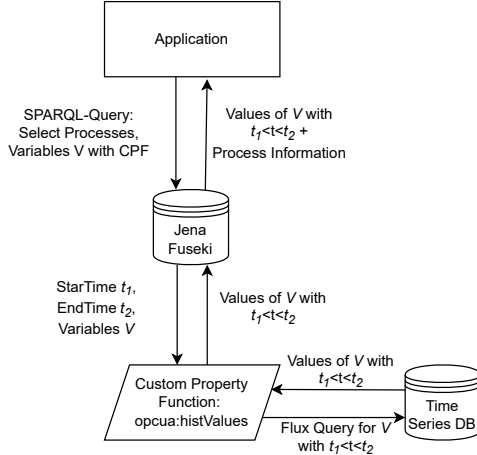


Fig. 5: Information flow of a query including the CPF.

Technical resource, start time and end time of the process are the only required process description elements to access the recorded OPC UA variable values. However, a more granular description of the process (e.g. only processes that were passed a specific parameter as an input) would allow for more precise filtering.

V. VALIDATION

In order to assess whether the knowledge graph and the access to the time series database meets the requirements from Section III, SPARQL queries were written to answer the competency questions from Table I and Table II.

The first query (s. Listing 1) provides a list of all variables, which belong to a certain machine in the ISA88 information model (CQ1.1).

```
SELECT ?NodeId ?BrowseName ?Time ?Value
WHERE {
  # 1. Selecting Machine in OPC UA-Information Model
  ?urnid owl:sameIndividualAs OpcSS:FullMachineTool.

  # 2. Selecting Variables from OpcUa-Model
  ?urnid OpcUa:hasComponent* ?node.
  ?node OpcUa:browseName ?BrowseName.
  ?node OpcUa:nodeId ?NodeId.
  ?node OpcUa:typeDefinition ?nodeclass.
  FILTER( ?nodeclass = "BaseDataVariableType" ||
    ?nodeclass = "FiniteStateVariableType" ||
    ?nodeclass = "AnalogUnitRangeType" ).

  # 3. Extracting Variable Values via CPF
  ?node OpcUa:histValues ( ?Time ?Value
    "2022-02-28T09:00:00Z"
    "2022-02-28T09:10:00Z" ).
}
ORDER BY ASC(?time)
```

Listing 1: SPARQL query to answer CQ 1.1 and 1.2 for a machine that follows OPC UA for Machine Tools Companion Specifications. Prefix declaration was omitted for brevity.

To do this, it navigates the OPC UA information model and extracts all variables whose type description fits the proper categories. If the line *?node OpcUa:histValues (?Time ?Value "starttime" "endtime")* is included, the query will also trigger the custom property function described in Section IV. Once executed, it constructs a query and sends it to InfluxDB to access all variable value changes by any variable that fits the selection criteria defined in the first part of the query. This way, the query also answers CQ1.2.

Time	Value	NodeId	BrowseName
2022-02-28T09:00:54Z	true	"ns=7;i=56510"	"IsRotating"
2022-02-28T09:01:26Z	true	"ns=7;i=56519"	"Locked"
2022-02-28T09:01:73Z	false	"ns=7;i=56510"	"IsRotating"
2022-02-28T09:01:70Z	"H2"	"ns=7;i=56600"	"UtilityName"
...	...	...	...

Table III: Exemplary SPARQL response to CQ 1.2.

It then returns the recorded value changes of all variables along with the timestamps, *NodeIds* and human readable *BrowseName* as shown in Table III. It should be noted that in the example the query accesses events. However, since InfluxDB does not differentiate between time series and events, the approach is equally valid for both types of time based data.

The queries for CQ 2.1 and 2.2 (see Listings 2 and 3) build upon the functionality shown in Listing 1.

```
SELECT ?Time ?Value ?NodeId ?BrowseName ?Process
WHERE {
  # 1. selecting ISA88-Procedure and -Unit:
  ?proc ISA88:isRealizedInProcessStage ?Process.
  ?unit VDI3682:isAssignedTo ?Process.
  FILTER( ?proc = OpcSS:UnitProcedure1 ).
  FILTER( ?unit = OpcSS:FullMachineTool ).

  # 2. selecting Timestamps of Processes
  ?Process VDI3682:hasInput ?stimeDE.
  ?stimeDE DINEN61360:hasTypeDescription
    OpcSS:StartTimeProcess;
    DINEN61360:hasInstanceDescription /
    DINEN61360:Value ?starttime.
  ?Process VDI3682:hasOutput ?etimeDE.
  ?etimeDE DINEN61360:hasTypeDescription
    OpcSS:EndTimeProcess;
    DINEN61360:hasInstanceDescription /
    DINEN61360:Value ?endtime.
  ?Process a VDI3682:Process.

  # 3. selecting Variables from OpcUa-Model
  ?urnid owl:sameIndividualAs ?unit.
  ?urnid OpcUa:hasComponent* ?node.
  ?node OpcUa:browseName ?BrowseName;
  OpcUa:typeDefinition ?nodeclass;
  OpcUa:nodeId ?NodeId.
  FILTER( ?nodeclass = "BaseDataVariableType" ||
    ?nodeclass = "FiniteStateVariableType" ||
    ?nodeclass = "AnalogUnitRangeType" ).

  # 4. Extracting Variable Values via CPF
  ?node OpcUa:histValues (?Time ?Value
    ?starttime ?endtime ).
}
ORDER BY ASC(?Time)
```

Listing 2: SPARQL query to answer CQ 2.1 by filtering for Unit Procedure. Prefix declaration was omitted for brevity.

They expanded by a query that extracts start and end timestamps from processes that can then be passed to the *OpcUa:histValues*-function. This way, only variables between the timestamps are returned.

The query can be adapted to filtering processes by any type of input, output or Technical Resource. This would also allow to filter by different information elements (e.g. process parameters) that function as input or output of the process.

The ISA88-information model offers another possibility to filter the selected processes by certain conditions with regards to its physical or procedural model. Since the information model contains explicit information about the type of process, it is possible to only select realizations of specific *Unit Procedures* or *Operations*.

The SPARQL query in Listing 2 extracts the start and end time from any process that is a realization of a specific *Unit Procedure* and executed by a specific *Unit*. It then retrieves the Units' variables via the OPC UA information model and queries the variable value changes from InfluxDB through the Custom Property Function *OpcUa:histValues()*.

With minor alterations in the first part, the same query can be used to answer CQ 2.2. As shown in Listing 3, only three lines in the first part need to be changed, while the remaining part of the query stays the same.

```
SELECT ?Time ?Value ?NodeId ?BrowseName ?Process
WHERE {
  # 1. selecting Article and -Unit:
  ?proc    ISA88:isRealizedInProcessStage ?Process.
  ?unit    VDI3682:isAssignedTo ?Process.
  ?Process VDI3682:hasOutput ?Product.
  ?Product OpcSS:hasProductType ?article.
  FILTER( ?article = OpcSS:Article1 ).
  FILTER( ?unit = OpcSS:FullMachineTool ).

  # 2. selecting Timestamps of Processes
  ?Process VDI3682:hasInput ?stimeDE.
  ?stimeDE DINEN61360:hasTypeDescription
            OpcSS:StartTimeProcess;
            DINEN61360:hasInstanceDescription /
            DINEN61360:Value ?starttime.
  ?Process VDI3682:hasOutput ?etimeDE.
  ?etimeDE DINEN61360:hasTypeDescription
            OpcSS:EndTimeProcess;
            DINEN61360:hasInstanceDescription /
            DINEN61360:Value ?endtime.
  ?Process a VDI3682:Process.

  # 3. selecting Variables from OpcUa-Model
  ?urnid   owl:sameIndividualAs ?unit.
  ?urnid   OpcUa:hasComponent* ?node.
  ?node    OpcUa:browseName ?BrowseName;
           OpcUa:typeDefinition ?nodeclass;
           OpcUa:nodeId ?NodeId.
  FILTER( ?nodeclass = "BaseDataVariableType" ||
           ?nodeclass = "FiniteStateVariableType" ||
           ?nodeclass = "AnalogUnitRangeType" ).

  # 4. Extracting Variable Values via CPF
  ?node    OpcUa:histValues (?Time ?Value
                             ?starttime ?endtime).
}
ORDER BY ASC(?Time)
```

Listing 3: SPARQL query to answer CQ 2.2 by filtering for Product Type. Query sections 2., 3., and 4. remain the same as in Listing 2.

Other filter criteria like selecting all sub processes that belong to a specific recipe are possible as well. This has the added benefit that filtering can be applied using the ODPs of the Formalized Process Description and ISA88, which are easily understandable for end users.

An example of the SPARQL response to either query can be seen in Table IV. It contains the value and timestamp of a node along with the process that was executed during the event. Grouping the result by process would allow for an easy creation of event traces for a process.

Time	Value	NodeId	BrowseName	Process
2022-02-28T09:00:54Z	true	"ns=7;i=56510"	"IsRotating"	Process 1
2022-02-28T09:01:26Z	true	"ns=7;i=56519"	"Locked"	Process 1
2022-02-28T09:02:73Z	false	"ns=7;i=56510"	"IsRotating"	Process 1
...	...	...	...	...
2022-02-28T10:03:36Z	true	"ns=7;i=56510"	"IsRotating"	Process 4
2022-02-28T10:04:11Z	true	"ns=7;i=56519"	"Locked"	Process 4
...	...	...	...	...

Table IV: Exemplary SPARQL response to CQ 2.1 and CQ 2.2.

VI. CONCLUSION

In this paper, a semantic model was presented that allows access to all recorded variable value changes from an OPC UA server based on a Formalized Process Description. To achieve this, the variable changes are logged in a time series data base and can then be accessed through a custom property function in a SPARQL query to the graph data base. Since the knowledge graph contains information about the production assets, the procedures they can perform and the processes that were realized (i.e. from an MES), this allows for the extraction of individual event traces of specific processes. The extracted event traces can then be used to analyse the timing information of processes down to the level of individual variable value changes. The approach was successfully validated on exemplary competency questions, which were answered by a number of SPARQL queries.

While the approach can answer the competency questions, some directions for future work remain. So far the prior knowledge is modeled based on the ISA 88 industry standard for batch control. The connection between process types, process instances and Technical Resources are described using other terminology in other domains. To broaden the scope of this approach, this information could also be modeled using skill based approaches [23] or the information models outlined in the VDI 5600 - Manufacturing Execution Systems [24].

The knowledge about the processes that were executed are currently materialized in the triple store. At a production facility that has an MES, this kind of data would usually be stored inside a dedicated relational database. To avoid duplication of data, this database could be directly accessed using an OBDA-tool like Ontop.

ACKNOWLEDGMENT

This work has been partially supported and funded by the German Federal Ministry of Education and Research (BMBF)

for the project "Time4CPS - A Software Framework for the Analysis of Timing Behaviour of Production and Logistics Processes" under the contract number 01IS20002. It was partially developed within the Fraunhofer Cluster of Excellence "Cognitive Internet Technologies".

REFERENCES

- [1] A. Bunte, P. Wunderlich, N. Moriz, P. Li, A. Mankowski, A. Rogalla, and O. Niggemann, "Why symbolic ai is a key technology for self-adaptation in the context of cpps," in *24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2019, pp. 1701–1704.
- [2] A. Maier, O. Niggemann, and Jens Eickmeyer, "On the learning of timing behavior for anomaly detection in cyber-physical production systems," *26th International Workshop on Principles of Diagnosis (DX-2015)*, 2015.
- [3] J. Ladiges, A. Fülber, E. Arroyo, A. Fay, C. Haubeck, and W. Lamersdorf, "Learning material flow models for manufacturing plants from data traces," in *IEEE 13th International Conference on Industrial Informatics (INDIN)*, 2015, pp. 294–301.
- [4] B. Vogel-Heuser, F. Ocker, I. Weiß, R. Mieth, and F. Mann, "Potential for combining semantics and data analysis in the context of digital twins," *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 379, no. 2207, p. 20200368, 2021.
- [5] V. Jirkovsky, M. Obitko, and V. Marik, "Understanding data heterogeneity in the context of cyber-physical systems integration," *IEEE Transactions on Industrial Informatics*, vol. 13, no. 2, pp. 660–667, 2017.
- [6] G. F. Schneider, "Semantic modelling of control logic in automation systems - knowledge-based support of the engineering and operation of control logic in building and industrial automation systems," Ph.D. dissertation, 2019.
- [7] S. Yin and O. Kaynak, "Big data for modern industry: Challenges and trends [point of view]," *Proceedings of the IEEE*, vol. 103, no. 2, pp. 143–146, 2015.
- [8] G. Xiao, L. Ding, B. Cogrel, and D. Calvanese, "Virtual knowledge graphs: An overview of systems and use cases," *Data Intelligence*, vol. 1, no. 3, pp. 201–223, 2019.
- [9] R. Schiekofner, S. Grimm, M. M. Brandt, and M. Weyrich, "A formal mapping between opc ua and the semantic web," in *IEEE 17th International Conference on Industrial Informatics (INDIN)*. Piscataway, NJ: IEEE, 2019, pp. 33–40.
- [10] C. Hildebrandt, A. Köcher, C. Küstner, C.-M. Lopez-Enriquez, A. W. Müller, B. Caesar, C. S. Gundlach, and A. Fay, "Ontology building for cyber-physical systems: Application in the manufacturing domain," *IEEE Transactions on Automation Science and Engineering*, vol. 17, no. 3, pp. 1266–1282, 2020.
- [11] G. Kalaycı, I. Grangel-Gonzalez, F. Lösch, G. Xiao, A. ul Mehdi, E. Kharlamov, and D. Calvanese, "Semantic integration of bosch manufacturing data using virtual knowledge graphs," *Proc. of the 19th Int. Semantic Web Conf. (ISWC)*, 2020.
- [12] F. Ekaputra, M. Sabou, E. Serral, E. Kiesling, and S. Biffl, "Ontology-based data integration in multi-disciplinary engineering environments: A review," *Open Journal of Information Systems (OJIS)*, vol. 4, pp. 1–26, 10 2017.
- [13] G. Steindl, T. Frühwirth, and W. Kastner, "Ontology-based opc ua data access via custom property functions," in *24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2019, pp. 95–101.
- [14] S. G. Mathias, S. Schmied, and D. Großmann, "An investigation on database connections in opc ua applications." Elsevier, 2020, pp. 602–609.
- [15] G. Steindl and W. Kastner, "Query performance evaluation of sensor data integration methods for knowledge graphs," in *Big Data, Knowledge and Control Systems Engineering (BdKCSE)*, 2019, pp. 1–8.
- [16] OPC Foundation, "OPC UA for Machinery Part 1: Basic Building Blocks," OPC Foundation, Scottsdale, AZ, Standard, May 2022.
- [17] International Society of Automation, "Ansi/isa–88.00.01: Batch control part 1: Models and terminology," Standard, May 2010.
- [18] VDI/VDE—Verein Deutscher Ingenieure e.V./Verband der Elektrotechnik, Elektronik, Informationstechnik, "VDI/VDE 3682-1: Formalised process descriptions - Concept and graphic representation," Verein Deutscher Ingenieure, Düsseldorf, D, Standard, May 2015.
- [19] Deutschen Institut für Normung DIN, "Standard data element types with associated classification scheme Part 1: Definitions - Principles and methods," Deutschen Institut für Normung DIN, Düsseldorf, D, Standard, Jul. 2017.
- [20] C. Hildebrandt, A. Scholz, A. Fay, T. Schröder, T. Hadlich, C. Diedrich, M. Dubovy, C. Eck, and R. Wiegand, "Semantic modeling for collaboration and cooperation of systems in the production domain," in *24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2019, pp. 1–8.
- [21] C. von Arnim, G. Görisch, T. Heinemann, Dennis, C. Ruckstetter, and H. Leinweber, "umati/sample-server: v0.3.0," Mar. 2022. [Online]. Available: <https://doi.org/10.5281/zenodo.6336935>
- [22] OPC Foundation, "OPC UA for Machine Tools Part 1: Monitoring and Job," OPC Foundation, Scottsdale, AZ, Standard, May 2020.
- [23] A. Köcher, C. Hildebrandt, L. M. Vieira da Silva, and A. Fay, "A formal capability and skill model for use in plug and produce scenarios," in *2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, vol. 1, 2020, pp. 1663–1670.
- [24] VDI—Verein Deutscher Ingenieure e.V., "VDI 5600-1: Manufacturing Execution Systems – MES," Verein Deutscher Ingenieure, Düsseldorf, D, Standard, Jan. 2015.